```
import tensorflow
tensorflow.__version__
```

```
'2.18.0'
```

1.Data Collection

1.1 Import necessary Libraries

```
import numpy as np #used for computing numerical calculations
import pandas as pd#When working with tabular data, such as data stored in spreadsheets or databases,
```

1.2 Load the Dataset

```
dataset = pd.read_csv("Churn_Modelling.csv")
```

```
dataset.head()
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMeml
0	1	15634602	Hargrave	619	France	Female	42.0	2.0	0.00	1	1	
1	2	15647311	Hill	608	Spain	Female	41.0	1.0	83807.86	1	0	
2	3	15619304	Onio	502	France	Female	42.0	8.0	159660.80	3	1	
3	4	15701354	Boni	699	France	Female	39.0	1.0	0.00	2	0	
4	5	15737888	Mitchell	850	Spain	Female	43.0	2.0	125510.82	1	1	

```
dataset.shape
```

```
(500, 14)
```

2.Data processing

2.1 Checking for null values

Unsupported Cell Type. Double-Click to inspect/edit the content.

```
#returns true that that particular column conatins null vaue
dataset.isnull().any()
```

```
RowNumber          False
CustomerId         False
Surname            False
CreditScore        False
Geography           True
Gender              True
Age                 True
Tenure              True
Balance            False
NumOfProducts      False
HasCrCard          False
IsActiveMember     False
EstimatedSalary    False
Exited             False
dtype: bool
```

```
#returns  the count of null values present in particular col
dataset.isnull().sum()
```

```
RowNumber          0
CustomerId         0
Surname            0
CreditScore        0
Geography          1
Gender             2
Age                2
Tenure             1
Balance            0
NumOfProducts      0
HasCrCard          0
IsActiveMember     0
EstimatedSalary    0
Exited             0
dtype: int64
```

Handling missing values

```
dataset["Age"]=dataset["Age"].fillna(dataset["Age"].mean())
dataset["Geography"]=dataset["Geography"].fillna(dataset["Geography"].mode()[0])
dataset["Gender"]=dataset["Gender"].fillna(dataset["Gender"].mode()[0])
dataset["Tenure"]=dataset["Tenure"].fillna(dataset["Tenure"].mode()[0])
```

```
dataset.isnull().any()
```

```
RowNumber          False
CustomerId         False
Surname            False
CreditScore        False
Geography          False
Gender             False
Age                False
Tenure             False
Balance            False
NumOfProducts      False
HasCrCard          False
IsActiveMember     False
EstimatedSalary    False
Exited             False
dtype: bool
```

```
dataset.head()
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMeml
0	1	15634602	Hargrave	619	France	Female	42.0	2.0	0.00	1	1	
1	2	15647311	Hill	608	Spain	Female	41.0	1.0	83807.86	1	0	
2	3	15619304	Onio	502	France	Female	42.0	8.0	159660.80	3	1	
3	4	15701354	Boni	699	France	Female	39.0	1.0	0.00	2	0	
4	5	15737888	Mitchell	850	Spain	Female	43.0	2.0	125510.82	1	1	

```
Start coding or generate with AI.
```

```
dataset.head(10)
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMeml
0	1	15634602	Hargrave	619	France	Female	42.0	2.0	0.00	1	1	
1	2	15647311	Hill	608	Spain	Female	41.0	1.0	83807.86	1	0	
2	3	15619304	Onio	502	France	Female	42.0	8.0	159660.80	3	1	
3	4	15701354	Boni	699	France	Female	39.0	1.0	0.00	2	0	
4	5	15737888	Mitchell	850	Spain	Female	43.0	2.0	125510.82	1	1	
5	6	15574012	Chu	645	Spain	Male	44.0	8.0	113755.78	2	1	
6	7	15592531	Bartlett	822	France	Male	50.0	7.0	0.00	2	1	
7	8	15656148	Obinna	376	Germany	Female	29.0	4.0	115046.74	4	1	
8	9	15792365	He	501	France	Male	44.0	4.0	142051.07	2	0	
9	10	15592389	H?	684	France	Male	27.0	2.0	134603.88	1	1	

2.2 Split the dependent and independent variables

```
X = dataset.iloc[:,3:13]
y = dataset.iloc[:,13:14]
```

2.3 Encoding

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
X["Gender"] = le.fit_transform(X["Gender"])
```

```
x.head(10)
```

	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary
0	619	France	0	42.0	2.0	0.00	1	1	1	101348.88
1	608	Spain	0	41.0	1.0	83807.86	1	0	1	112542.58
2	502	France	0	42.0	8.0	159660.80	3	1	0	113931.57
3	699	France	0	39.0	1.0	0.00	2	0	0	93826.63
4	850	Spain	0	43.0	2.0	125510.82	1	1	1	79084.10
5	645	Spain	1	44.0	8.0	113755.78	2	1	0	149756.71
6	822	France	1	50.0	7.0	0.00	2	1	1	10062.80
7	376	Germany	0	29.0	4.0	115046.74	4	1	0	119346.88
8	501	France	1	44.0	4.0	142051.07	2	0	1	74940.50
9	684	France	1	27.0	2.0	134603.88	1	1	1	71725.73

One hot encode Geography column

```
X = pd.get_dummies(X, columns=['Geography'], drop_first=True)
```

```
X.head()
```

	CreditScore	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Geography_Germany	Gec
0	619	0	42.0	2.0	0.00	1	1	1	101348.88	False	
1	608	0	41.0	1.0	83807.86	1	0	1	112542.58	False	
2	502	0	42.0	8.0	159660.80	3	1	0	113931.57	False	
3	699	0	39.0	1.0	0.00	2	0	0	93826.63	False	
4	850	0	43.0	2.0	125510.82	1	1	1	79084.10	False	

```
X.shape
```

```
(500, 11)
```

```
type(X)
```

```
pandas.core.frame.DataFrame
```

```
y
```

	Exited
0	1
1	0
2	1
3	0
4	0
...	...
495	0
496	0
497	0
498	0
499	0

500 rows × 1 columns

```
X.shape
```

```
(500, 11)
```

```
y.shape
```

```
(500, 1)
```

Unsupported Cell Type. Double-Click to inspect/edit the content.

2.4 Train test split

Unsupported Cell Type. Double-Click to inspect/edit the content.

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size = 0.2,random_state = 0)
#100000 , 20000, 80000
```

```
X_train.shape
```

```
(400, 11)
```

```
X_test.shape
```

```
(100, 11)
```

```
X_train.head(5)
```

	CreditScore	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Geography_Germany	G
107	785	0	36.0	2.0	99806.85	1	0	1	36976.52	True	
336	659	0	32.0	3.0	150923.74	2	0	1	174652.51	True	
71	813	1	29.0	6.0	0.00	1	1	0	33953.87	False	
474	571	0	33.0	1.0	0.00	2	1	0	102750.70	False	
6	822	1	50.0	7.0	0.00	2	1	1	10062.80	False	

2.5 Feature scaling

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

```
x_train
```

```
array([[ 0.16958176, -1.09168714, -0.46488237, ...,  1.10643166,
        -0.5698444 ,  1.74367544],
       [-2.30455945,  0.91601335,  0.30085148, ..., -0.74866447,
         1.75486502, -0.57350122],
       [-1.19119591, -1.09168714, -0.94346603, ...,  1.48533467,
        -0.5698444 , -0.57350122],
       ...,
       [ 0.9015152 ,  0.91601335, -0.36916564, ...,  1.41231994,
        -0.5698444 , -0.57350122],
       [-0.62420521, -1.09168714, -0.08201545, ...,  0.84432121,
        -0.5698444 ,  1.74367544],
       [-0.28401079, -1.09168714,  0.87515187, ...,  0.32472465,
         1.75486502, -0.57350122]])
```

3.Model Building – Artificial Neural Network

3.1 Import TensorFlow Libraries

```
from keras.models import Sequential  # initializing the model
from keras.layers import Dense,Input
```

3.2 Initialize ANN Model

```
model = Sequential([
    Input(shape=(X_train.shape[1],)),
    Dense(11, activation="relu"),
    Dense(6, activation="relu"),
    Dense(6, activation="relu"),
    Dense(1, activation="sigmoid")
])
```

```
model.summary()
```

```
Model: "sequential_10"
```

Layer (type)	Output Shape	Param #
dense_32 (Dense)	(None, 11)	132
dense_33 (Dense)	(None, 6)	72
dense_34 (Dense)	(None, 6)	42
dense_35 (Dense)	(None, 1)	7

```
Total params: 253 (1012.00 B)
Trainable params: 253 (1012.00 B)
```

3.3 Compile the Model

```
model.compile(optimizer = "adam", loss = "binary_crossentropy",metrics = ["accuracy"])
```

3.4 Train the Model

```
history = model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=32,validation_data=(X_test, y_test)
)
```

```
Epoch 1/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 3s 33ms/step - accuracy: 0.7102 - loss: 0.6765 - val_accuracy: 0.7800 - val_loss: 0.6244
Epoch 2/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7347 - loss: 0.6563 - val_accuracy: 0.7900 - val_loss: 0.6063
Epoch 3/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8054 - loss: 0.6268 - val_accuracy: 0.8000 - val_loss: 0.5897
Epoch 4/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7903 - loss: 0.6282 - val_accuracy: 0.8000 - val_loss: 0.5740
Epoch 5/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8141 - loss: 0.5923 - val_accuracy: 0.8000 - val_loss: 0.5575
Epoch 6/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7975 - loss: 0.5846 - val_accuracy: 0.8000 - val_loss: 0.5436
Epoch 7/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7852 - loss: 0.5747 - val_accuracy: 0.8000 - val_loss: 0.5299
Epoch 8/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8181 - loss: 0.5348 - val_accuracy: 0.8000 - val_loss: 0.5158
Epoch 9/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8306 - loss: 0.4992 - val_accuracy: 0.8000 - val_loss: 0.5027
Epoch 10/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7875 - loss: 0.5387 - val_accuracy: 0.8000 - val_loss: 0.4938
Epoch 11/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8132 - loss: 0.4927 - val_accuracy: 0.8000 - val_loss: 0.4843
Epoch 12/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7999 - loss: 0.4911 - val_accuracy: 0.8000 - val_loss: 0.4775
Epoch 13/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.7997 - loss: 0.4837 - val_accuracy: 0.8000 - val_loss: 0.4712
Epoch 14/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8042 - loss: 0.4651 - val_accuracy: 0.8000 - val_loss: 0.4662
Epoch 15/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8073 - loss: 0.4558 - val_accuracy: 0.8000 - val_loss: 0.4621
Epoch 16/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8166 - loss: 0.4460 - val_accuracy: 0.8000 - val_loss: 0.4583
Epoch 17/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7994 - loss: 0.4600 - val_accuracy: 0.8000 - val_loss: 0.4545
Epoch 18/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7782 - loss: 0.4855 - val_accuracy: 0.8000 - val_loss: 0.4501
Epoch 19/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8011 - loss: 0.4378 - val_accuracy: 0.8000 - val_loss: 0.4470
Epoch 20/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.7957 - loss: 0.4419 - val_accuracy: 0.8000 - val_loss: 0.4438
Epoch 21/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8018 - loss: 0.4191 - val_accuracy: 0.8000 - val_loss: 0.4412
Epoch 22/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7945 - loss: 0.4398 - val_accuracy: 0.8000 - val_loss: 0.4382
Epoch 23/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.7862 - loss: 0.4458 - val_accuracy: 0.8000 - val_loss: 0.4361
Epoch 24/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.8056 - loss: 0.4099 - val_accuracy: 0.8000 - val_loss: 0.4330
Epoch 25/100
```

```
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8168 - loss: 0.3989 - val_accuracy: 0.8000 - val_loss: 0.4309
Epoch 26/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7783 - loss: 0.4364 - val_accuracy: 0.8000 - val_loss: 0.4280
Epoch 27/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7821 - loss: 0.4260 - val_accuracy: 0.8000 - val_loss: 0.4253
Epoch 28/100
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7726 - loss: 0.4296 - val_accuracy: 0.8000 - val_loss: 0.4225
Epoch 29/100
```

```
batch size =32
data size =x_train=400
400/32 approximately 13
13 records at a time
```

```
  Cell In[773], line 1
    batch size =32
          ^
SyntaxError: invalid syntax
```

3.5 Test the Model

```
ypred = model.predict(X_test)
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
```

```
ypred[0]
```

```
array([0])
```

```
y_test
```

	Exited
90	1
254	0
283	0
445	0
461	0
...	...
372	0
56	0
440	0
60	0
208	1

100 rows × 1 columns

```
# o to 1 , >0.5 = 1 , <0.5 , 0
```

```
ypred = np.where(ypred >0.5 ,1 , 0)
```

```
ypred[0]
```

```
array([0])
```

4. Model Evaluation

```
loss, accuracy = model.evaluate(X_test, y_test)

print("\nTest Loss:", loss)
print("Test Accuracy:", accuracy)
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8292 - loss: 0.3644

Test Loss (MSE): 0.3884824514389038
Test MAE: 0.4100000023841858
```

```
Start coding or generate with AI.
```

```
from sklearn.metrics import accuracy_score
accuracy_score(y_test, ypred)
```

```
0.81
```

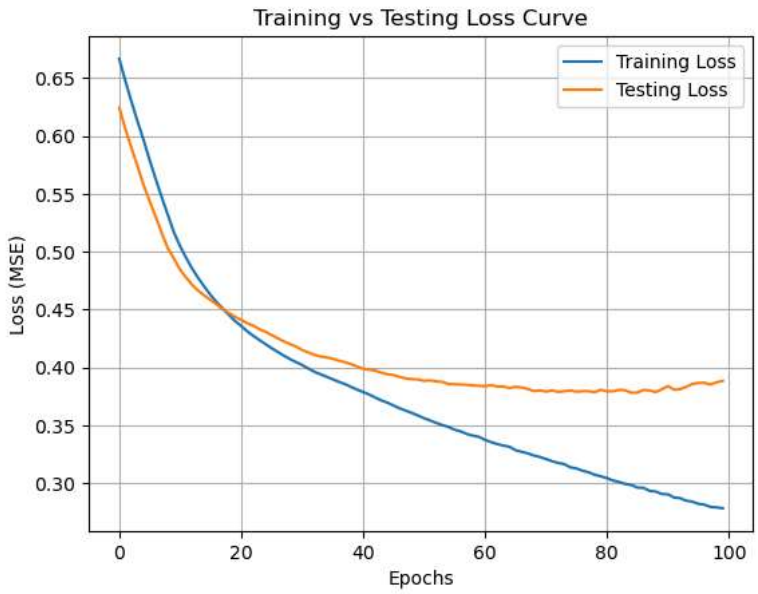
Plot Training vs Testing Loss

```
import matplotlib.pyplot as plt
plt.figure()

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title("Training vs Testing Loss Curve")
plt.xlabel("Epochs")
plt.ylabel("Loss (MSE)")
plt.legend(["Training Loss", "Testing Loss"])

plt.grid(True)
plt.show()
```



5.Random Value Prediction

```
X.head(2)
```

	CreditScore	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary	Geography_Germany	Geog
0	619	0	42.0	2.0	0.00	1	1	1	101348.88	False	
1	608	0	41.0	1.0	83807.86	1	0	1	112542.58	False	

```
model.predict(sc.transform([[608,0,41.0,1.0,083807.86,1,0,1,112542.58,0,1]]))
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 47ms/step
C:\Users\CHETAN KUMAR\anaconda3\Lib\site-packages\sklearn\base.py:493: UserWarning: X does not have valid feature names, but Sta
  warnings.warn(
array([[0.06809705]], dtype=float32)
```